

Devoir surveillé d'informatique

⚠ Consignes

- Les programmes demandés doivent être écrits en C. On suppose que les bibliothèques standard usuelles (`<stdio.h>`, `<stdlib.h>`, `<stdbool.h>`, `<assert.h>`, ...) sont déjà importées.
- On pourra toujours librement utiliser une fonction demandée à une question précédente, même si cette question n'a pas été traitée.
- Veillez à présenter vos idées et vos réponses partielles, même si vous ne trouvez pas la solution complète à une question.
- La clarté et la lisibilité de la rédaction et des programmes sont des éléments de notation.

□ Exercice 1 : Questions de cours

On considère l'algorithme suivant :

Algorithme : Multiplier sans utiliser *

Entrées : $n \in \mathbb{N}, m \in \mathbb{N}$

Sorties : nm

```

1  r ← 0
2  tant que m > 0 faire
3      |   m ← m - 1
4      |   r ← r + n
5  fin
6  return r

```

- Q1–** Donner les valeurs successives prises par les variables m , n et r si l'on exécute cet algorithme avec $n = 7$ et $m = 4$. On pourra recopier et compléter le tableau suivant :

	n	m	r
valeurs initiales	7	4	0
après un tour de boucle	7	3	7
après deux tours de boucle	7	2	14
après trois tours de boucle	7	1	21
après quatre tours de boucle	7	0	28

- Q2–** Donner une implémentation de cet algorithme en langage C sous la forme d'une fonction `multiplie` de signature `int multiplie(int n, int m)`. On précisera soigneusement la spécification de cette fonction en commentaire dans le code et on vérifiera les préconditions à l'aide d'instructions `assert`.

```

1  // Prend en entrée deux entiers positifs n et m et renvoie leur produit nm
2  int multiplie(int n, int m)
3  {
4      assert(n >= 0 && m >= 0);
5      int r = 0;
6      while (m > 0)
7      {
8          m = m - 1;
9          r = r + n;
10     }
11     return r;
12 }

```

- Q3–** Donner la définition d'un variant de boucle, puis prouver que cet algorithme termine.

Un *variant de boucle* est une quantité qui dépend des variables du programme et qui est :

1. entière,
2. positive,
3. strictement décroissante.

Dans l'algorithme ci-dessus, la quantité m est un variant de boucle ; en effet :

1. $m \in \mathbb{N}$ (par précondition).
2. m reste positif ou nul : par condition d'entrée dans la boucle $m \geq 1$, et comme on décrémente m de 1, m reste ≥ 0 après le tour de boucle.
3. m décroît strictement car il est diminué de 1 lors de chaque passage dans la boucle.

L'algorithme termine car on a trouvé un variant de boucle.

Q4– Donner la définition d'un invariant de boucle, puis prouver que cet algorithme est correct.

Un *invariant de boucle* est une propriété qui dépend des variables du programme et qui :

1. est vraie avant d'entrer dans la boucle (initialisation),
2. reste vraie après un tour de boucle si elle l'était au début du tour (conservation).

En sortie de boucle, la validité d'un invariant permet de prouver la correction de l'algorithme.

En notant m_0 la valeur initiale de m , montrons que la propriété I : « $r = (m_0 - m)n$ » est un invariant de boucle.

1. Avant d'entrer dans la boucle, $m = m_0$ donc $(m_0 - m)n = 0$. Comme r est initialisé à 0, la propriété I est vérifiée.
2. On suppose I vérifiée au début d'un tour de boucle et on note r' (resp. m') les valeurs prises par r (resp. m) à la fin de ce tour. On a $m' = m - 1$ et $r' = r + n$. Calculons : $(m_0 - m')n = (m_0 - (m - 1))n = (m_0 - m + 1)n = (m_0 - m)n + n$. Or, par hypothèse de récurrence (I vraie), $(m_0 - m)n = r$. Donc $(m_0 - m')n = r + n = r'$. La propriété I est donc conservée.

En sortie de boucle, la condition $m > 0$ est fausse, donc $m = 0$. L'invariant nous assure alors que $r = (m_0 - 0)n = m_0n$. L'algorithme calcule bien le produit attendu et est donc correct.

□ Exercice 2 : Retourner un tableau

Dans cet exercice, on s'intéresse à un algorithme permettant de « retourner » un tableau, c'est-à-dire réorganiser l'ordre de ses éléments de façon que le premier élément devienne le dernier, le deuxième devienne l'avant-dernier et ainsi de suite. Par exemple, le tableau $\{2, 7, 1, 9, 3\}$ deviendrait $\{3, 9, 1, 7, 2\}$. On notera bien que le retournement doit s'effectuer *en place*, c'est-à-dire qu'on ne veut pas créer de nouveau tableau mais simplement modifier le tableau existant. En notant t_0 le tableau initial et t_r le tableau « retourné », on a donc pour tout $k \in \llbracket 0; n - 1 \rrbracket$, $t_r[k] = t_0[n - 1 - k]$. On propose pour cela l'algorithme suivant :

Algorithme : Retourner un tableau

Entrées : Un tableau t de taille n

Sorties : Aucune

Résultat : Le tableau t est modifié (retourné)

```

1  $i \leftarrow 0$ 
2  $j \leftarrow n - 1$ 
3 tant que  $j - i > 0$  faire
4   | échanger les éléments d'indices  $i$  et  $j$  dans  $t$ 
5   |  $i \leftarrow i + 1$ 
6   |  $j \leftarrow j - 1$ 
7 fin
```

Q5– Montrer que cet algorithme termine.

Dans le cas où le tableau est vide, l'algorithme termine directement sans entrer dans la boucle **while**. Sinon,

- $j - i$ est entier comme différence de deux entiers
- $j - i$ est positif avant d'entrer dans la boucle ($n \geq 1$) et reste positif par condition d'entrée dans la boucle
- $j - i$ décroît strictement puisque i est incrémenté et j décrémenté

Q6– Montrer que la propriété suivante, notée I , est un invariant de l'algorithme : $i + j = n - 1$.

- Avant d'entrer dans la boucle, $i = 0$ et $j = n - 1$, donc I est vraie.
- On suppose I vraie au début d'un tour de boucle. Montrons qu'alors I est conservée à la fin de ce passage. En notant $i' = i + 1$ et $j' = j - 1$ les nouvelles valeurs, on a $i' + j' = i + 1 + j - 1 = i + j$. Or, par hypothèse $i + j = n - 1$, donc $i' + j' = n - 1$ et I est conservée.

Q7– Montrer que cet algorithme est correct. On pourra pour cela considérer l'invariant I' : « le tableau t contient les valeurs de t_r (le tableau retourné) pour tous les indices $k \in \llbracket 0; i - 1 \rrbracket \cup \llbracket j + 1; n - 1 \rrbracket$ » et utiliser l'invariant I de la question précédente.

On vérifie que l'invariant proposé est vrai :

- Avant d'entrer dans la boucle, $\llbracket 0; i - 1 \rrbracket \cup \llbracket j + 1; n - 1 \rrbracket = \emptyset$ et donc I' est vraie par vacuité.
- Conservé lors d'un passage dans la boucle : si I' est vrai en entrant dans la boucle, alors on échange $t[i]$ avec $t[j]$. D'après l'invariant I démontré à la question précédente, $j = n - 1 - i$. On échange donc $t[i]$ avec $t[n - 1 - i]$. Comme, par hypothèse, les éléments situés avant i et après j sont déjà correctement échangés (I' supposé vrai), la propriété I' reste vraie pour les indices s'étendant jusqu'à i et j .

En sortie de boucle, on a $j - i \leq 0$ et donc $\llbracket 0; i - 1 \rrbracket \cup \llbracket j + 1; n - 1 \rrbracket = \llbracket 0; n - 1 \rrbracket$; le tableau t contient bien les valeurs du tableau retourné.

Q8– On souhaite utiliser cet algorithme afin d'écrire en langage C une fonction **retourner_str** qui inverse une chaîne de caractères (par exemple, la chaîne "MP2I" devient "I2PM"). Rappeler la façon dont sont implémentées en C les chaînes de caractères et expliquer pour quelle raison il n'est pas utile de fournir la longueur de la chaîne en paramètre de la fonction **retourner_str**.

En C, les chaînes de caractères sont des tableaux de caractères se terminant par le caractère sentinelle `'\0'`. On peut donc obtenir la taille d'une chaîne en recherchant la position de ce caractère ; il n'est donc pas utile de la passer en paramètre.

Q9– Écrire une fonction de signature `void echange(char s[], int i, int j)` qui échange dans s les caractères situés aux indices i et j .

```

1 void echange(char s[], int i, int j)
2 {
3     // Échange les caractères situés aux indices i et j dans s
4     char temp = s[i];
5     s[i] = s[j];
6     s[j] = temp;
7 }
```

Q10– Écrire une implémentation de la fonction **retourner_str** sans utiliser les fonctions de la bibliothèque `<string.h>`.

```

1 void retourner_str(char s[])
2 {
3     // Inverse en place la chaîne de caractères s
4     int n = 0;
5     while (s[n] != '\0')
6     {
7         n++;
8     }
9     int i = 0;
10    int j = n - 1;
11    while (j - i > 0)
12    {
13        echange(s, i, j);
14        i = i + 1;
15        j = j - 1;
16    }
17 }

```

Q11– On rappelle qu'un palindrome est un mot qui se lit de la même façon de droite à gauche ou de gauche à droite, par exemple « *radar* » est un palindrome, mais « *tata* » n'en est pas un. Pour tester si un mot est un palindrome, un élève propose la solution suivante :

```

1 bool palindrome(char s[])
2 {
3     //renvoie true ssi s est un palindrome
4     char copie[] = s;
5     retourner_str(copie);
6     return s==copie;
7 }

```

Ce programme ne compile pas et produit une erreur à la ligne 4 : « *error : invalid initializer* ». Expliquer la cause de cette erreur (on ne demande pas de réécrire la fonction `palindrome`, mais d'expliquer de façon succincte l'erreur commise à la ligne 4 et la façon d'y remédier).

On ne peut pas affecter directement un tableau en C, on doit travailler élément par élément. Il faudrait donc copier un à un les caractères de `s` dans le tableau `copie`.

Q12– On suppose corrigée l'erreur commise à la ligne 4. Cette fonction est-elle alors correcte ? Justifier votre réponse.

Non, cette fonction n'est pas correcte. En C, l'expression `s == copie` compare les adresses mémoire des deux tableaux (qui sont distinctes) et non leurs contenus : le test sera donc toujours faux. De plus, il n'existe pas d'opérateur d'égalité globale entre tableaux ; on doit comparer les caractères un à un dans une boucle (ou utiliser `strcmp`).

□ Exercice 3 : Autour de la recherche de minimums

On souhaite écrire une fonction de signature `int min(int tab[], int taille)` qui prend en entrée un tableau non vide d'entiers `tab` ainsi que sa taille `taille` et renvoie le minimum des éléments de ce tableau. On donne ci-dessous deux réponses d'élèves où les fonctions sont nommées respectivement `min_eleve1` et `min_eleve2`.

```
1  int min_eleve1(int tab[], int taille)
2  {
3      // Renvoie le minimum des éléments de tab (supposé non vide)
4      assert(taille > 0);
5      int cmin = 0;
6      for (int i = 0; i < taille; i++)
7      {
8          if (tab[i] < cmin)
9          {
10             cmin = tab[i];
11          }
12      }
13      return cmin;
14 }
```

```
1  int min_eleve2(int tab[], int taille)
2  {
3      // Renvoie le minimum des éléments de tab (supposé non vide)
4      assert(taille > 0);
5      int cmin = tab[0];
6      for (int i = 0; i < taille - 1; i++)
7      {
8          if (tab[i + 1] < tab[i])
9          {
10             cmin = tab[i + 1];
11          }
12      }
13      return cmin;
14 }
```

Q13– Donner un exemple de tableau à 3 éléments pour lequel `min_eleve1` renvoie un résultat correct.

On peut prendre par exemple le tableau {5, -2, 1} pour lequel la fonction renvoie bien -2. En effet, le problème de cette fonction est l'initialisation à 0 du minimum : dès lors que le minimum est négatif ou nul, la valeur renvoyée est correcte.

Q14– Proposer un exemple de votre choix montrant que `min_eleve1` ne répond pas à sa spécification.

Si le tableau ne contient que des valeurs *strictement positives*, alors, le minimum étant initialisé à 0, le test de la ligne 8 n'est jamais vrai et la valeur renvoyée pour le minimum sera 0. On peut par exemple prendre le tableau {2, 7, 5} : son minimum est 2, mais la fonction renvoie 0.

Q15– Donner un exemple de tableau à 3 éléments pour lequel `min_eleve2` renvoie un résultat correct.

On peut prendre par exemple le tableau {5, -2, -3} pour lequel la fonction renvoie bien -3. En effet, le problème de cette fonction est de comparer l'élément avec son suivant et non avec le minimum trouvé jusque-là.

Q16– Proposer un exemple de votre choix montrant que `min_eleve2` ne répond pas à sa spécification.

On peut prendre par exemple le tableau {-4, -2, -3} pour lequel la fonction renvoie -3. En effet, la comparaison entre les deux derniers éléments met à jour `cmin` à -3, alors que le minimum est -4.

Q17– Donner une version de la fonction `min` conforme à sa spécification.

```
1  int min(int tab[], int taille)
2  {
3      // Renvoie le minimum des éléments de tab (supposé non vide)
4      assert(taille > 0);
5      int cmin = tab[0];
6      for (int i = 1; i < taille; i++)
7      {
8          if (tab[i] < cmin)
9          {
10             cmin = tab[i];
11          }
12      }
13      return cmin;
14 }
```

Q18– On veut maintenant obtenir, en plus du minimum, son nombre d'occurrences dans le tableau. Par exemple, pour le tableau {1, 0, 7, 2}, le minimum est 0 (il apparaît 1 fois) et pour {1, 7, -3, 2, -3, 4}, le minimum est -3 (il apparaît 2 fois). Pour cela, on propose d'écrire une fonction de signature : `int min_occ(int tab[], int taille, int *nb)`. Cette fonction renvoie le minimum *et* modifie la valeur pointée par `nb` afin qu'elle contienne le nombre d'apparitions du minimum. Donner une implémentation en langage C de la fonction `min_occ`.

```
1  int min_occ(int tab[], int taille, int *nb)
2  {
3      // Renvoie le minimum et affecte à *nb son nombre d'occurrences
4      assert(taille > 0);
5      int cmin = tab[0];
6      *nb = 1;
7      for (int i = 1; i < taille; i++)
8      {
9          if (tab[i] < cmin)
10         {
11             cmin = tab[i];
12             *nb = 1;
13         }
14         else if (tab[i] == cmin)
15         {
16             *nb = *nb + 1;
17         }
18     }
19     return cmin;
20 }
```

On s'intéresse maintenant à la recherche des deux plus petites valeurs d'un tableau contenant au moins deux éléments (ces deux valeurs pouvant être égales si le minimum apparaît au moins deux fois).

Q19– Écrire un algorithme (en pseudo-langage) permettant de résoudre ce problème. On écrira une instruction `return` pour renvoyer les deux plus petites valeurs.

Algorithme : Deux minimums**Entrées :** Un tableau t de n entiers avec $n \geq 2$.**Sorties :** Les deux plus petites valeurs de t

```

1 si  $t[0] < t[1]$  alors
2   |  $m1 \leftarrow t[0]$ 
3   |  $m2 \leftarrow t[1]$ 
4 fin
5 sinon
6   |  $m1 \leftarrow t[1]$ 
7   |  $m2 \leftarrow t[0]$ 
8 fin
9 pour  $i \leftarrow 2$  à  $n - 1$  faire
10  | si  $t[i] < m1$  alors
11  |   |  $m2 \leftarrow m1$ 
12  |   |  $m1 \leftarrow t[i]$ 
13  | fin
14  | sinon si  $t[i] < m2$  alors
15  |   |  $m2 \leftarrow t[i]$ 
16  | fin
17 fin
18 return  $m1, m2$ 

```

Q20– En C, on ne peut pas directement renvoyer deux entiers, aussi on propose de passer à la fonction deux pointeurs vers des entiers dont le contenu sera mis à jour afin de contenir après appel les deux plus petites valeurs. Écrire cette fonction.

```

1 void deuxmin(int tab[], int taille, int *min1, int *min2)
2 {
3     // Affecte à *min1 et *min2 les deux plus petites valeurs de tab
4     assert(taille > 1);
5     if (tab[0] < tab[1]) {
6         *min1 = tab[0];
7         *min2 = tab[1];
8     } else {
9         *min1 = tab[1];
10        *min2 = tab[0];
11    }
12    for (int i = 2; i < taille; i++) {
13        if (tab[i] < *min1) {
14            *min2 = *min1;
15            *min1 = tab[i];
16        } else if (tab[i] < *min2) {
17            *min2 = tab[i];
18        }
19    }
20 }

```

Q21– Proposer un invariant de boucle qui permettrait de prouver la correction de cet algorithme. On ne demande pas de faire une preuve de correction, mais simplement d'énoncer un invariant préservé durant le déroulement de l'algorithme.

L'invariant à prouver est qu'au début de chaque itération d'indice i (pour $2 \leq i \leq n$), $m1$ et $m2$ contiennent les deux plus petites valeurs du sous-tableau $t[0 \dots i - 1]$ avec $m1 \leq m2$. En fin d'algorithme, puisque $i = n$, ces deux variables contiennent bien les deux plus petites valeurs de l'ensemble du tableau, ce qui garantit la correction.